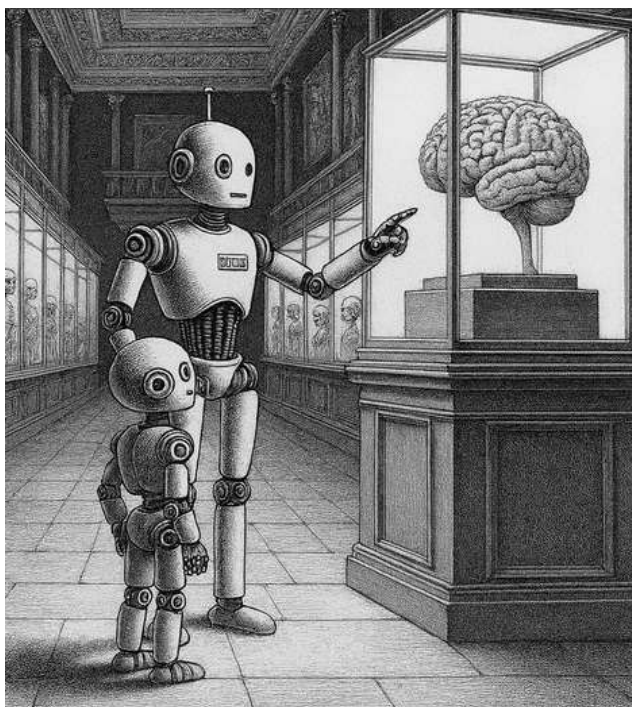
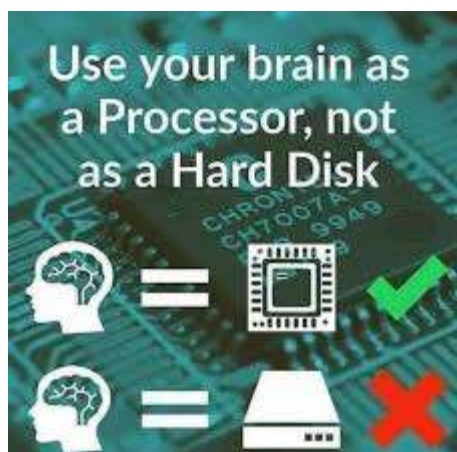


ЕЗИЦИ И СРЕДИ ЗА ПРОГРАМИРАНЕ

1. Преговор

- А) Разлика между информация и данни? Хардуер и софтуер?
- Б) Функции на процесора и харддиска. Прилики, разлики, принципи на работа?



And that is the original processor!"

MY BRAIN WHEN...

DAYTIME



NIGHT



GAMING



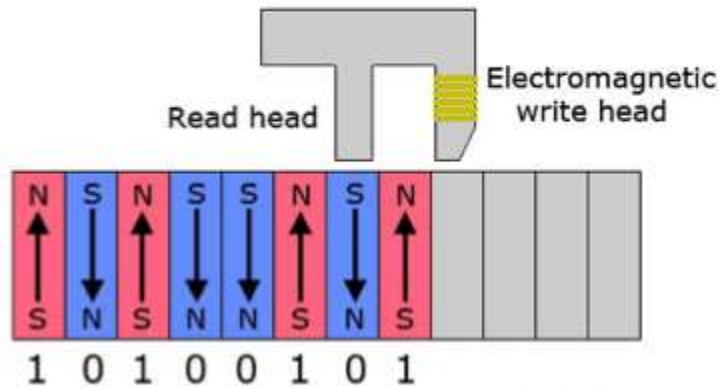
TALKING
TO MY CRUSH



STUDYING

```
#include <iostream> and <iostream.h> are used to print output  
to your console  
  
using namespace std;  
  
int main() {  
    // Print the first five even and odd natural numbers  
    // using for loop  
    int i = 1;  
    while(i <= 10) {  
        if(i % 2 == 0) {  
            cout << "Even: " << i << endl;  
        } else {  
            cout << "Odd: " << i << endl;  
        }  
        i++;  
    }  
    return 0;  
}
```

Hard drive read/write head



"If real is what you can feel, smell, taste and see, then 'real' is simply electrical signals interpreted by your brain."

<https://www.rapidtables.com/convert/number/binary-to-ascii.html>

В) Операционни системи – прилики, разлики Windows / Mac / Linux (Unix)?

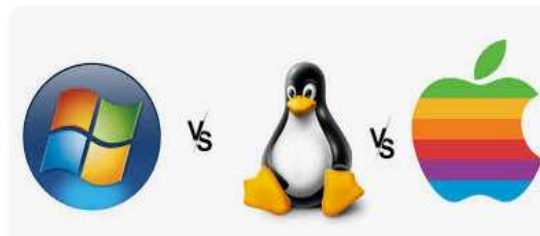
Windows – Бил Гейтс, със силно развит GUI – graphic user interface (съответствие на формите), с много диалогови прозорци, но реалната визуализация забавя и е малка част от кода. „Истинският“ Windows е регистъра regedit. Потребителски-ориентиран, затова доминира при PC – personal computers

Linux – Линус Торвалд, доминиращата ОС при сървърите, силно развит команден интерфейс, видимост върху всичко и голям контрол.

MacOS – базирана на Unix, много защитена, безотказно работи, скъпа

Unix – най-старата от използваните в момента, доразвива се до Linux, използва се за ИИ и суперкомпютри

DOS – Disk OS – наподобява CMD



Г) Интерфейс (диалогови прозорци) и команди CMD с вградени функции.

Д) Процедури = текстови файлове с поредица от команди .bat

Е) **Разлика между писане на код и програмиране?** – алгоритмите

* програмиране – (if, else, for...) C, C++, Java, Python...

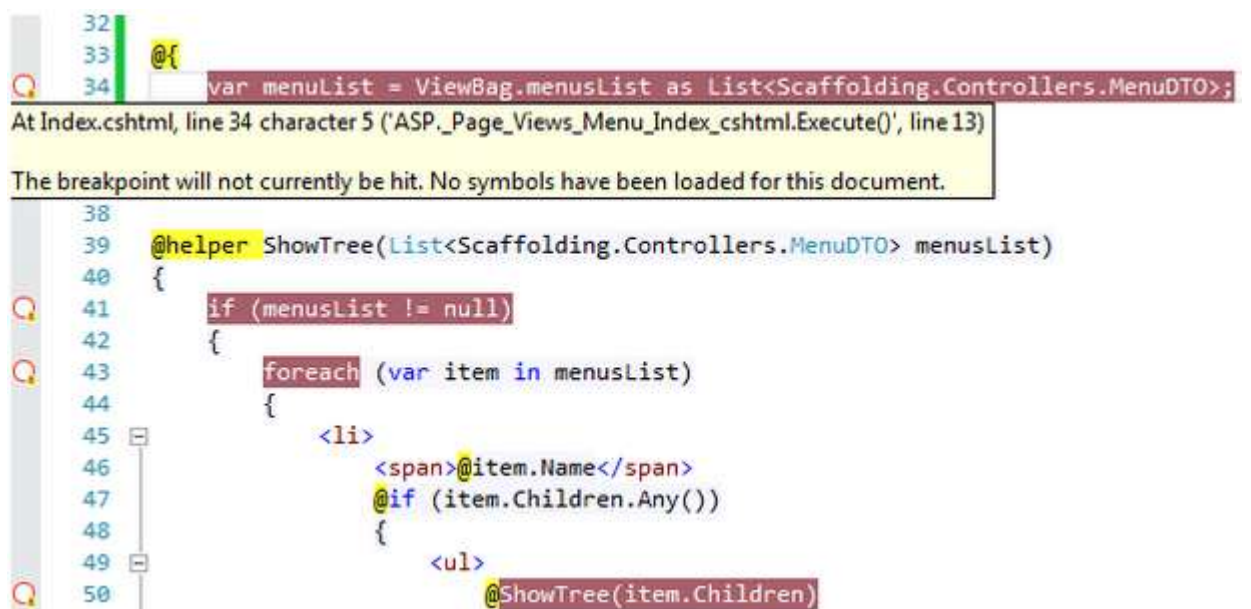
* писане на код – (ключови думи <>) HTML, CSS, SQL...


```

<html>
  <head>
    <title>My Page Title</title>
  </head>
  <body>
    <script type="text/javascript">
    </script>
    <div>
      <h1>This is browser DOM</h1>
    </div>
  </body>
</html>

```

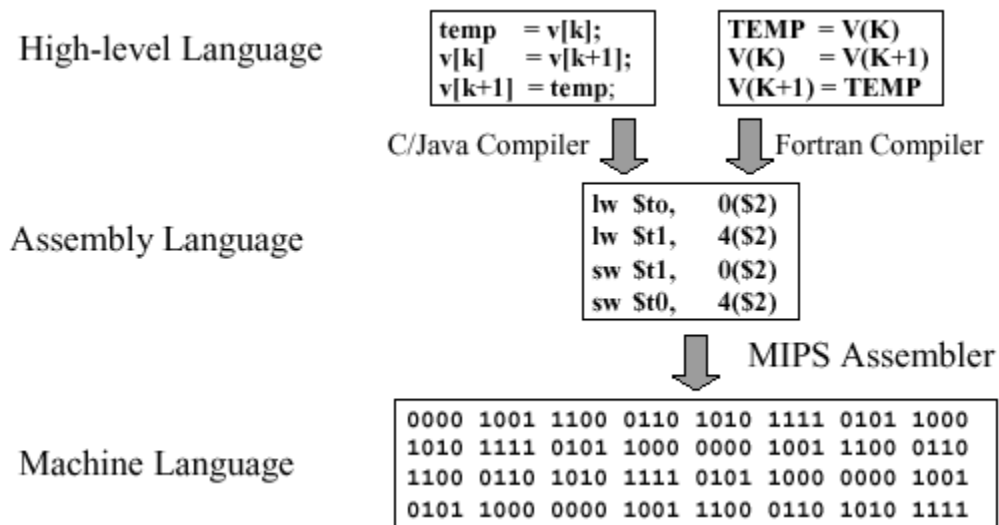
* блендиране (смесване) на 2 езика като C# + HTML = .cshtml



2. Езици за програмиране

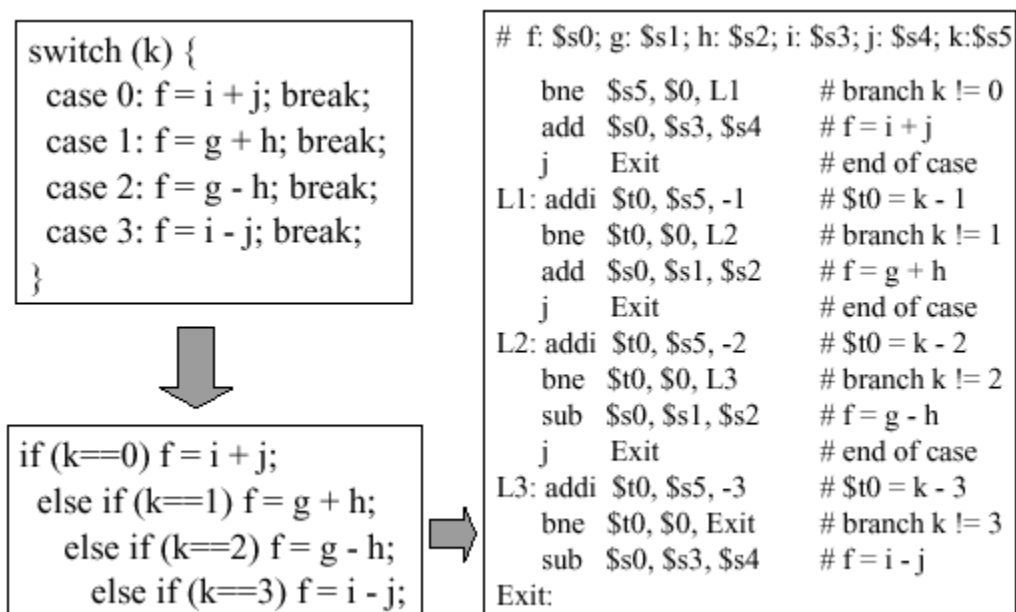
Синтаксис – правилата

Семантика – смисъл

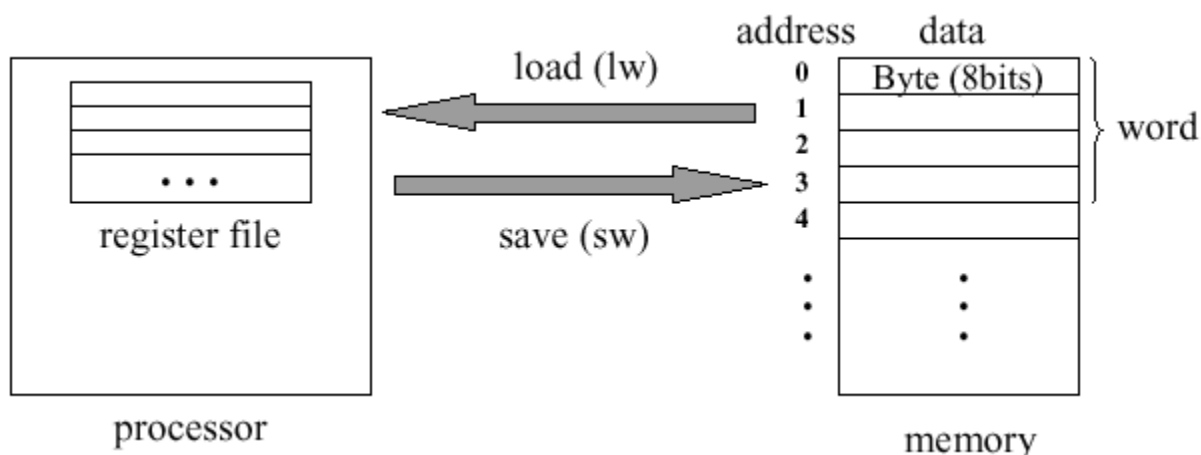


Машинен език – на най-ниско ниво стоят само 0 и 1. На по-високо ниво има и други цифри.

Асемблер – съдържа съкращения на английски думи като ADD, MOV – Много близо до хардуера, вирусите се пишат на асемблер, както и драйвърите



Превод на програма от високо ниво на асемблер, след което се записва в паметта на още по-ниско ниво:



Ниво	Език	Превод	Характеристики	Приложения
Ниско	Машинен, Асемблер	Интерпретатор	Максимална скорост и контрол, много сложни, зависят от хардуера	Драйвери, firmware, системно програмиране
Средно	C, C++, Rust	Компилятор	Висока скорост, баланс между контрол и абстракция	Операционни системи, игри, софтуер с висока производителност
Високо	Python, Java, C#	Виртуална машина / Интерпретатор	Лесни за учене, по-бавни	Уеб приложения, data science, AI, бърз прототипинг
Много високо	SQL, MATLAB, WOLFRAM	Специализиран локален домейн	Не са универсални	Заявки към бази данни, математически и научни изчисления

Транслаторът - проверява за синтактични грешки и след отстраняването им превежда програма, написана на език от високо ниво на машинен език. Бива 2 вида – компилатор и интерпретатор.

- **Компилятор:** превежда целия програмен код на машинен език преди изпълнение, създавайки отделен изпълним файл. Компилираните програми са по-бързи, но грешките се откриват при компилация. Езици C++, Java...
- **Интерпретатор:** превежда и изпълнява кода ред по ред по време на работа, без да създава отделен изпълним файл. Интерпретираните програми са по-бавни, но грешките се откриват по време на изпълнение. Езици Python, PHP, JS
- **Виртуална машина (Java):** Това е "хибридният" подход. Кодът се компилира до междинен bytecode, който се изпълнява от виртуална машина (JVM). Това прави езика преносим "write once, run anywhere".



- **1843 г** – първата програма – Ада Лъвлейс (първият програмист в историята) – създава алгоритъм за изчисляване на числата на Бернули за аналитичната машина на Чарлз Бабидж.
- **1950-60: Фокус върху машината** – Fortran (FORmula TRANslator - за научни изчисления), COBOL (за бизнес), LISP (за AI). Всички са създадени за конкретни домейни.

- **1970-80: Фокус върху методите** – C се ражда за да напишат UNIX. Въвежда концепциите за структурно програмиране и пряк контрол върху хардуера. **Pascal**, **Delphi**. Какво ви говорят имената?
- **1990-2000: Фокус върху софтуерния инженеринг** – **C++** (разширение на C с ООП), **Java** (улесняване на C++), **Python** (четимостта и продуктивността са на първо място). През 1993 г – HTML от Тим Бърнърс-Лий – създава първия сървър и интернет.
- **2000-днес: Фокус върху специфични проблеми** – **JavaScript** (за уеб), **C#** (за Windows ecosystem), **Go** (за мрежови услуги и успоредност), **Rust** (за безопасност на паметта и скорост).

Великолепните 4: Python, LISP, C, Perl -

Python – изглежда лесен, доста бавен, но мощен, ООП (с обекти от реалния свят / класове)

C – не е подходящ за първи език, но е езикът на UNIX

Perl – динамични уеб приложения и системно администриране

LISP – дава дълбоки познания, дори никога да не се наложи да го използваме (функционално програмиране за ИИ – какъв резултат искаме, а не как да се постигне)

3. Среди за програмиране

IDE – Integrated Development Environment:

- * текстов редактор – пишем текст
- * компилатор / интерпретатор – превежда кода на машинен език
- * дебъгер – инструмент за намиране на грешки
- * Intellisense – автодопълване
- * Garbage Collector – система за управление на паметта
- * VCS – Version Control System – система за проследяване на промените – най-популярна е Git

В нашия случай – Visual Studio